Group symbol:

Team: **NOVA**

Project title:

SCoop

Student Cooperation (personalised matching and organising platform for building student's projects)

Team members (*filled by PM, Team Leader*):

No	Name	Surname	Student ID	Role
1	Nikita	Homaniuk	284209	*PM, Team Leader*
2	Oleksandr	Kvasha	284202	*Team member*
3	Vitalii	Kozak	288330	*Team member*
4	Andrii	Taftay	284205	*Team member*

SE 2026 Winter - Project Report S1

SCoop Concept

SCoop (**S**tudent **Coop**eration) is a web application platform whose functionality and ideas are designed to solve real student problems. The key problem that inspired SCoop is the student desire to implement their ideas and skills in personal projects or different events like hackathons and competitions which are highly dependent on group work and having a complete team.

In many cases, a student's own social bubble is simply not enough to find like-minded companions with the same vision or the needed skills to build a project the way they would like to. For example, if you need a team of 4-5 people for an upcoming hackathon, most of your close friends might be busy or have the exact same skills as you. At the same time, formal university scientific clubs are often too bureaucratic, locked to a single faculty, and force you to work on their pre-selected topics rather than your own ideas. Posting on Discord or Facebook groups also doesn't work well because messages get buried in minutes and you can't filter people properly.

To solve this problem, our group wants to create a web app whose key functionality is based on the ability to not only create a project with the right position roles, tags and goals or find such a project to join, but also to build a smart system that helps match the right people based on their skills and vision.

How SCoop Works in Practice:

- **Detailed Student Profiles:** Skills, preferred roles, experience level, goals, reputation, location, university, and weekly free time - every essential piece of information provided in a user's profile helps both project creators and participants find people who match their intersecting interests.
- **Project "Casting" with Open Role Slots:** When a student creates a project on SCoop, they don't just write a general post asking for help. Instead, they open specific role positions (for example: *1x Backend Dev, 1x Frontend Dev, 1x UI/UX Designer, 1x Presenter/PM*). Each position has its own required skills and expected time commitment.
- **Smart Matching System:** Instead of scrolling through hundreds of random posts, our system calculates a match percentage (0–100%) between a student's profile and an open project role. It checks not just technical skills, but also whether the student's vision matches the team's goal (for example, whether everyone wants to sweat hard to win 1st place at a hackathon or just build some side project for their portfolio). Project creators can sort applicants by this match score or browse student profiles and invite matching people directly.
- **Team Lock-In & Start:** When a project creator accepts someone for an open role, that specific position closes automatically. Once all positions are filled (for example 5 out of 5 members found), the team is officially formed, remaining applications are closed and all accepted members get access to the team's private workspace links (like Discord or GitHub) so they can start working right away.
- **Reputation & Trust:** Because students are teaming up with people outside their close friend group, SCoop uses university email verification and post-project teammate ratings. Finishing projects without dropping out builds up a student's reliability score on their profile, so everyone knows who they can count on.

What We Focus On (Our Scope): To make sure our 4-person team can build and test this cleanly during the semester, SCoop focuses entirely on **finding, matching and forming the right team**. We are not building another chat app or task board like Trello. Once SCoop brings the right people together, the team moves to their favorite tools to build the project and comes back to SCoop at the end to rate their teammates and add the finished project to their profile.

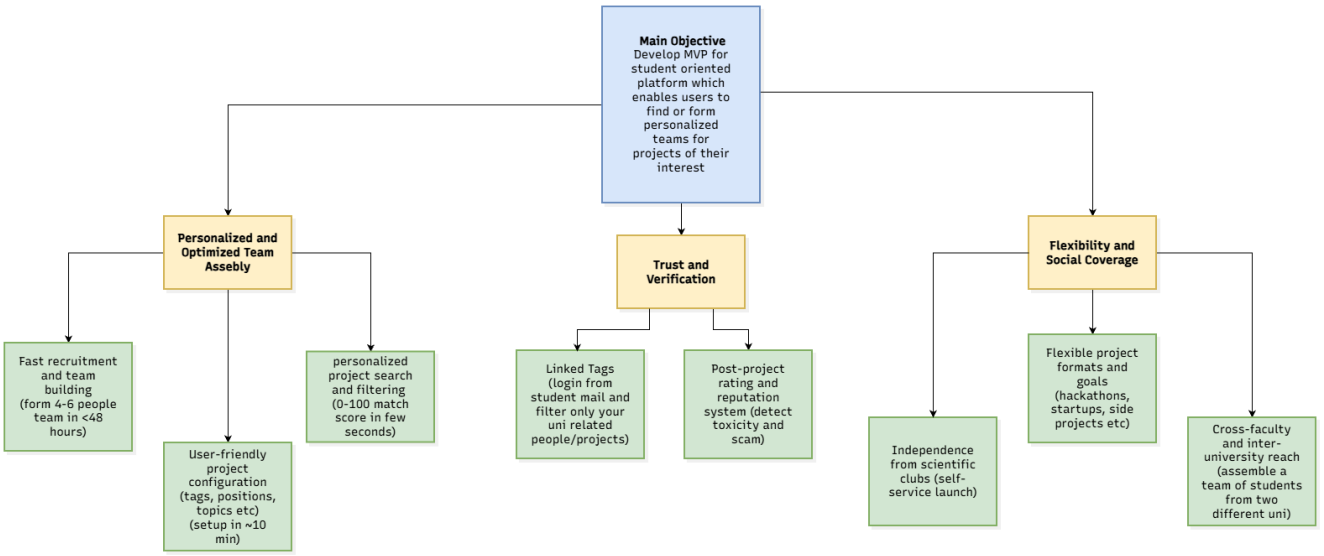
After creating our MVP we will certainly consider adding new tools by thinking which current external tools have disadvantages in use for our primary users.

1. Elaboration of application concept (S1)

1.1. Project (business) goals

Present as a tree of the project's objectives, i.e., decomposition of general project objectives into specific ones. A goal should be:

- relevant to the project,
- important from the perspective of the end-user,
- achievable,
- measurable,
- realistic (achievable in terms of constraints: cost and time-oriented, technological, organizational, human resources, legal)



SCoop goal hierarchy

Main goal: Help students with an idea or skill set end up in the right team, fast, so their projects actually happen.

1. Get the right people together, quickly

- Cut the time it takes to go from "I need a team" to "we have a team".
- Make sure the people who join are a good fit, not just whoever replied first. They should have complementary skills and want the same thing out of the project (win the hackathon vs side portfolio piece).
- Make it easy for a student to explain what they're building and who they need, in about 10 minutes, without writing a long post.
- Save people from scrolling through random posts. Finding a project or a teammate should take seconds and give a clear "this fits you" signal.

2. Make students feel safe teaming up with strangers

- Keep the community limited to real students, so you know who you're dealing with.
- Make people's reliability visible, so a person who finishes projects stands out from one who ghosts.
- Discourage toxic behavior and scams.
- Let people choose a community domain whether it's only students from their university or whole city.

3. Break out of the limits of a student's usual circle

- Free students from the social or academic bubbles by giving them access to a much bigger pool of people.
- Free students from the bureaucracy of scientific clubs: anyone can start a project without permission or a pre-set topic.
- Support all kinds of projects (hackathons, startups, side projects, competitions).

1.2. Identification of project's internal and external Stakeholders

Identify the role of each **internal** stakeholder in the project, along with a textual description.

Symbol	Name	Role	Description
SH1	Project manager	Coordinator	Manages communication within the team, plans and allocates tasks, tracks progress, gathers feedback from supervisor and verifies completeness.
SH2	Project team	Development and design	Develop, design, maintain and trace code, write the documentation and deliver the software in a complete form.
SH3	QA members	Testing	Test the software, report bugs, and verify that requirements are met, assess code to confirm meeting the standards
SH9	Real-life use testers	Real testing	Testing from external scope, provide on-hand experience before full deployment.
SH10	Course Supervisor (Instructor)	Feedback and design coordination	Provide feedback on project progress and correct the team's course during development and testing.

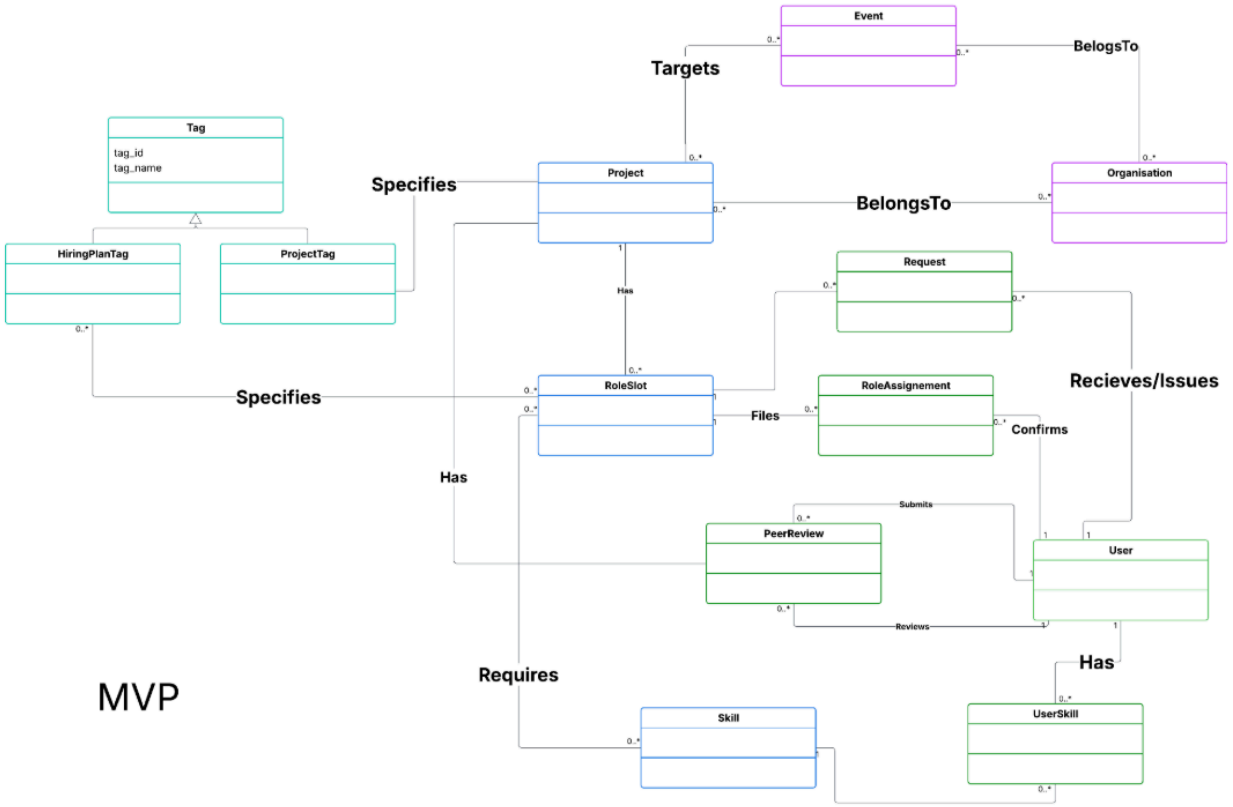
Identify the role of each **external** stakeholder in the project, along with a textual description.

Symbol	Name	Role	Description
SH4	Students initiators / applicants	Users	Use the software, and their needs and feedback shape features. Create/look up projects, request/apply for specific roles and assemble/join teams.
SH5	Universities & student organization	Sponsors & promoters	Promote the platform to encourage student collaboration, officially use it to actively promote it among potential user-base.
SH6	Tech companies & Mentors	Sponsors & Ext. Users	Invest in software to get access to paid features (recruit in own interests), propose real-world topics for projects.
SH7	Cloud infrastructure providers	Supplier	Supplies the resources and infrastructure needed for app operation, such as hosting and database service.
SH8	*Hackathon & event organizers*	*Partners and promoters*	*Promote upcoming hackathons on the platform or create specific events to draw in new users.*

1.3. Domain description

- Description of phenomena in the application domain that can be observed.
- Definitions of concepts that abstract these phenomena.
- The concepts include:
    - entities – things that one can “distinguish” from the fraction of reality,
    - relationships – either properties (unary relationships of entities) that yield some information about them or relationships between two or more entities (binary or *n*-ary relations),
    - events – things that can happen (occur).
- Use UML class diagram to depict entities (class with attributes), relationships (associations).
- This diagram **must** be supplemented with the list of entities, relationships and events along with their description in natural language.

SE 2026 Winter - Project Report S1



Entity Name	Description in Application Domain
User	A verified student registered on the platform with university credentials, maintaining personal skills and a public reputation score.
Project	A collaborative initiative, hackathon effort or academic project created by a student with defined goals, timeframe, and open roles.
RoleSlot	A discrete open position within a project (e.g., "Frontend Developer") specifying required skills, capacity and expected commitment.
Skill	A standardized competency dictionary item (e.g., Python, Figma, UI/UX) used to profile user strengths and role requirements.
UserSkill	A profile-level junction entity linking a user to a declared skill with self-assessed proficiency.
Request	A bidirectional recruitment transaction between a user and a role slot, handling inbound applications and outbound invitations.
RoleAssignment	An active membership record confirmed once a request is accepted, binding a student to a project role.

SE 2026 Winter - Project Report S1

Event	An external competition or hackathon that projects can optionally target to assemble teams before a registration deadline.
Organisation	An academic body, student club or partner entity that hosts events and classifies student user groups.
Tag	A general classification taxonomy, specialized into ProjectTag (thematic topics) and RoleTag (commitment and location conditions).
PeerReview	A post-project evaluation submitted by teammates upon project completion, scoring reliability to update public reputation.

Event id	Event	Description	Entities involved
User and profile			
E01	User registers	A student creates an account with personal data. Reputation of students starts with initial value.	User
E02	University email verified	The student confirms their account through a verification link sent to their university email address.	User
E03	Skill added to profile	A user declares a skill. A UserSkill record is created.	User, UserSkill, Skill
E04	Skill removed from profile	A user no longer declares a skill. A UserSkill record is updated.	User, UserSkill, Skill
E05	Role preference set	A user ranks a role they would like to take in projects. A UserRolePreference record is created.	User, UserRolePreference, Role
E06	Role preference changed or withdrawn	A user changes rank or removes a preferred role. validTo is set on the old preference, and a new one may be created.	UserRolePreference
Organization and event			
E07	Organization registered	A university, student organization, or company is added to the platform.	Organization
E08	Event created	An organization announces an event, such as a hackathon or competition. createdAt is set.	Event, Organization
E09	Event archived	An event ends or is no longer relevant, and archivedAt is set.	Event

Project			
E10	Project draft created	A user starts creating a project with a name and description. Its status is set to creating.	Project
E11	Project published	The project creator finishes setup and makes the project visible. Its status changes to created.	Project
E12	Project tagged	A descriptive tag is attached to the project.	Project, ProjectTag
E13	Project linked to event or organization	The project is registered under an event or an organization.	Project, Event, Organization
E14	Project finished	The team completes the project, and its status changes to finished. This enables peer reviews.	Project
E15	Project cancelled	The project is abandoned before completion, and its status changes to cancelled.	Project
E16	Project moved to maintenance	A finished project continues to be maintained, and its status changes to maintained.	Project
E17	Project archived	The project is removed from active listings, and archivedAt is set.	Project
Recruitment			
E18	Role slot opened	The project creator opens an open role slot with a defined role, required skills, and role tags (e.g., location, weekly commitment).	RoleSlot, Project, Skill, RoleTag
E19	Application sent	A user applies for an open role slot. A Request is created with direction 'application' and status 'pending'.	Request, User, RoleSlot
E20	Invitation sent	The project creator invites a matching user to a role slot. A Request is created with direction 'invitation' and status 'pending'.	Request, User, RoleSlot
E21	Request accepted	The receiving side accepts the application or invitation, and its status changes to accepted.	Request
E22	Request declined	The receiving side rejects the application or invitation, and its status changes to declined.	Request
E23	Member assigned to role	After an accepted request, the user is assigned to the role slot. A RoleAssignment is created.	RoleAssignment, User, RoleSlot

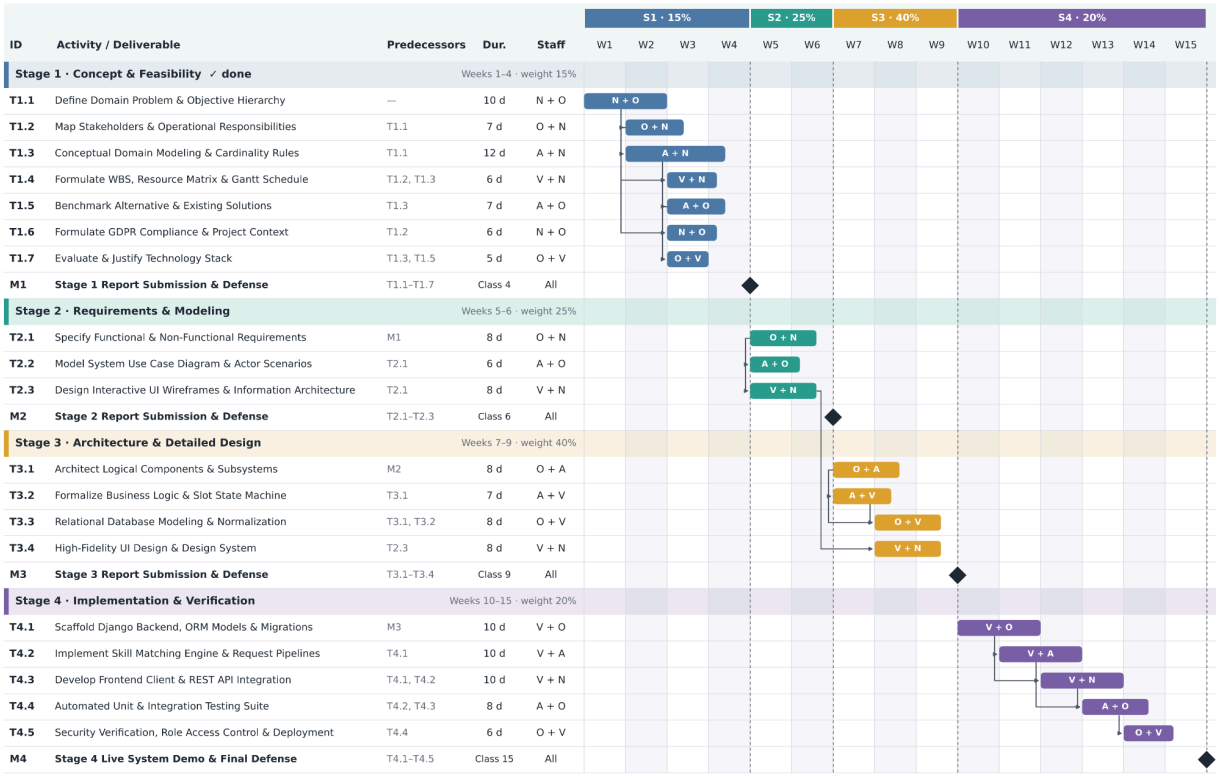
E24	Role slot closed	A RoleAssignment confirms the position, or the creator closes the slot manually. Remaining pending requests for that slot are resolved.	RoleSlot, Request
E25	Team formed	All open role slots of a project are filled. The team is complete, and members receive workspace access.	Project, RoleSlot, RoleAssignment
E26	Member leaves or is removed from team	A member drops out or is removed before the project ends. end_datetime is set on the RoleAssignment, and the slot can be reopened.	RoleAssignment
Reputation			
E27	Peer review submitted	After a project is finished, a team member rates another member with a mark and an optional comment.	PeerReview, User, Project
E28	Reputation updated	A user's reputation score is recalculated based on received peer reviews and completed projects without dropping out.	User, PeerReview, RoleAssignment

1.4. Project schedule (Gantt chart)

- tasks with dependencies,
- start time and duration,
- resource allocation: staff, costs, equipment.

SCoop – Project Schedule (Gantt Chart)

Team NOVA · SE 2026 Winter · bar length = duration in working days (1 week = 5 days)



ID	Activity / Deliverable	Predecessors	Dur.	Staff
Stage 1 · Concept & Feasibility ✓ done			Weeks 1–4 · weight 15%	
T1.1	Define Domain Problem & Objective Hierarchy	—	10 d	N + O
T1.2	Map Stakeholders & Operational Responsibilities	T1.1	7 d	O + N
T1.3	Conceptual Domain Modeling & Cardinality Rules	T1.1	12 d	A + N
T1.4	Formulate WBS, Resource Matrix & Gantt Schedule	T1.2, T1.3	6 d	V + N
T1.5	Benchmark Alternative & Existing Solutions	T1.3	7 d	A + O
T1.6	Formulate GDPR Compliance & Project Context	T1.2	6 d	N + O
T1.7	Evaluate & Justify Technology Stack	T1.3, T1.5	5 d	O + V
M1	**Stage 1 Report Submission & Defense**	T1.1–T1.7	Class 4	All
Stage 2 · Requirements & Modeling			Weeks 5–6 · weight 25%	
T2.1	Specify Functional & Non-Functional Requirements	M1	8 d	O + N
T2.2	Model System Use Case Diagram & Actor Scenarios	T2.1	6 d	A + O
T2.3	Design Interactive UI Wireframes & Information Architecture	T2.1	8 d	V + N
M2	**Stage 2 Report Submission & Defense**	T2.1–T2.3	Class 6	All
Stage 3 · Architecture & Detailed Design			Weeks 7–9 · weight 40%	
T3.1	Architect Logical Components & Subsystems	M2	8 d	O + A
T3.2	Formalize Business Logic & Slot State Machine	T3.1	7 d	A + V
T3.3	Relational Database Modeling & Normalization	T3.1, T3.2	8 d	O + V
T3.4	High-Fidelity UI Design & Design System	T2.3	8 d	V + N
M3	**Stage 3 Report Submission & Defense**	T3.1–T3.4	Class 9	All
Stage 4 · Implementation & Verification			Weeks 10–15 · weight 20%	
T4.1	Scaffold Django Backend, ORM Models & Migrations	M3	10 d	V + O
T4.2	Implement Skill Matching Engine & Request Pipelines	T4.1	10 d	V + A
T4.3	Develop Frontend Client & REST API Integration	T4.1, T4.2	10 d	V + N
T4.4	Automated Unit & Integration Testing Suite	T4.2, T4.3	8 d	A + O
T4.5	Security Verification, Role Access Control & Deployment	T4.4	6 d	O + V
M4	**Stage 4 Live System Demo & Final Defense**	T4.1–T4.5	Class 15	All

Staff: **N** = Nikita Homaniuk (PM) **O** = Oleksandr Kvasha **V** = Vitalii Kozak **A** = Andrii Taftay

Task (pair initials inside) Dependency (predecessor → successor) Milestone / stage gate (next stage starts after it)

Successors start in the last week of their predecessor (overlap); the arrow leaves the predecessor bar at the successor's start. Cost of every activity: €0.

SCoop – Resource Allocation (staff, equipment, cost)

ID	Activity / Deliverable	Staff (pair & roles)	Equipment & tools	Cost
Stage 1 · Concept & Feasibility · Weeks 1–4 · weight 15%				
T1.1	Define Domain Problem & Objective Hierarchy	Nikita (PM / Visionary) + Oleksandr (Logic)	Workstations, MS Teams	€0
T1.2	Map Stakeholders & Operational Responsibilities	Oleksandr (Tech Lead) + Nikita (PM Governance)	Workstations, Miro/Draw.io	€0
T1.3	Conceptual Domain Modeling & Cardinality Rules	Andrii (Deep Analysis/UML) + Nikita (PM/Scope)	Draw.io, GitHub Monorepo	€0
T1.4	Formulate WBS, Resource Matrix & Gantt Schedule	Vitalii (Execution Lead) + Nikita (PM Planning)	GitHub Projects, MS Teams	€0
T1.5	Benchmark Alternative & Existing Solutions	Andrii (Market Analysis) + Oleksandr (Tech)	Web Browsers, Documentation	€0
T1.6	Formulate GDPR Compliance & Project Context	Nikita (Governance/PM) + Oleksandr (Tech)	Legal Specs, Docs Suite	€0
T1.7	Evaluate & Justify Technology Stack	Oleksandr (Tech Decider) + Vitalii (Dev Lead)	Python/Django Tech Docs	€0
M1	**Stage 1 Report Submission & Defense (Class 4)**	All Team (Presented by Team Member)	Presentation Suite, MS Teams	€0
Stage 2 · Requirements & Modeling · Weeks 5–6 · weight 25%				
T2.1	Specify Functional & Non-Functional Requirements	Oleksandr (Spec Lead) + Nikita (PM Scope)	Docs Suite, GitHub Issues	€0
T2.2	Model System Use Case Diagram & Actor Scenarios	Andrii (UML/Analysis) + Oleksandr (Logic)	Draw.io, Visual Paradigm	€0
T2.3	Design Interactive UI Wireframes & Information Architecture	Vitalii (UI Prototyping) + Nikita (UX Vision)	Figma Edu, Web Browser	€0
M2	**Stage 2 Report Submission & Defense (Class 6)**	All Team (Defense by Appointed Member)	Presentation Suite, MS Teams	€0
Stage 3 · Architecture & Detailed Design · Weeks 7–9 · weight 40%				
T3.1	Architect Logical Components & Subsystems	Oleksandr (Tech Lead) + Andrii (Analysis)	Draw.io, PlantUML	€0
T3.2	Formalize Business Logic & Slot State Machine	Andrii (Logic Lead) + Vitalii (Execution)	PlantUML, Python Prototypes	€0
T3.3	Relational Database Modeling & Normalization	Oleksandr (Data Architect) + Vitalii (Dev)	PostgreSQL/SQLite, Draw.io	€0
T3.4	High-Fidelity UI Design & Design System	Vitalii (UI Implementation) + Nikita (PM UX)	Figma Education Suite	€0
M3	**Stage 3 Report Submission & Defense (Class 9)**	All Team (Defense by Appointed Member)	Presentation Suite, MS Teams	€0
Stage 4 · Implementation & Verification · Weeks 10–15 · weight 20%				
T4.1	Scaffold Django Backend, ORM Models & Migrations	Vitalii (Dev Lead) + Oleksandr (Data Review)	VS Code, Python 3.12, Django	€0
T4.2	Implement Skill Matching Engine & Request Pipelines	Vitalii (Algorithm Dev) + Andrii (Logic QA)	VS Code, Django REST	€0
T4.3	Develop Frontend Client & REST API Integration	Vitalii (Integration Lead) + Nikita (PM Test)	React/HTML5, Postman	€0
T4.4	Automated Unit & Integration Testing Suite	Andrii (QA Lead) + Oleksandr (Validation)	Pytest, Django Test Runner	€0
T4.5	Security Verification, Role Access Control & Deployment	Oleksandr (Security) + Vitalii (Deployment)	Localhost / Docker, GitHub CI	€0
M4	**Stage 4 Live System Demo & Final Defense (Class 15)**	All Team (Live Demo Execution)	Running SCoop Instance	€0
Total project cost				**€0**

1.5. Identification of existing or alternative solutions

Each competitor should be characterized with

- name of the project/application,
- main features,
- advantages,
- disadvantages.

№	Existing Alternative Solution	Features	Advantages	Disadvantages
1	Linkedin	Profiles, connections, groups, job listings	Large network; searchable skills and experience	Career-focused; No project availability
2	Discord	Community servers, text and voice channels, server roles	Large network, server roles can be used to tag skills and experience	Hard to discover suitable people and projects, server roles may lack skill details.
3	Facebook / Other Social Media	Groups, posts, messaging, events	Campus communities are familiar with it, it already has a large user base.	Posts get buried; skills, projects and availability are unstructured and hard to find suitable ones.
4	Major League Hacking	Hackathons, events, student developer programs	Strong student hackathon community	Less suited for finding collaborators for ongoing projects. Small number of hackathons outside of US (e.g. No hackathons in Poland)
5	Devpost	Hackathon listings, team formation, project showcases	Helps users find hackathons and present work	Mainly focused on hackathon projects and doesn't provide user matching by skills and other parameters.
6	Skillinkr	Student profiles, skill-based discovery, team matching	Has skill tags and a user matching system	Users cannot find and apply to hackathons directly. Non-functional now.
7	University Environment (scientific clubs, etc)	Depends on the University and its science clubs. (In PWr: student activity days)	Already exists and already has a lot students	Hard to quickly find projects that match interest. Hard to quickly find people with required skills.

1.6. Project context

Indicate aspects that are important for the product. It should lead to the identification of constraints of the project and the chances for its development.

- **Application context:**

SCoop operates directly inside the university extracurricular ecosystem, targeting hackathons, term projects and student initiatives which are usually spontaneous. Because student recruitment typically happens under pressure (often 48 to 72 hours before deadlines)the onboarding and project setup must take minutes, not days. Students have zero patience for administrative paperwork, but they quickly lose interest in flat, spreadsheet-like listings. SCoop balances this by using visual role slots and an instant compatibility score making team building feel intuitive and rewarding, even entertaining to some extent. Finally, the app makes users flexible in their established habits by letting them use other tools once the team is assembled, handing members directly off to Discord or GitHub rather than trapping them in a proprietary chat system.

- **Technological context:**

SCoop is a web application designed to work comfortably on both desktop computers and mobile phones, since students often browse projects and answer invitations on the go. The interface is planned to behave like a Single-Page Application (SPA), where moving between profiles, projects and role slots feels instant without full page reloads. The whole technology stack is built on free developer tiers and open-source infrastructure (Django, PostgreSQL, Tailwind CSS, Resend and GitHub), so the project can be developed and run without any financial costs. The matchmaking system that recommends people for open positions uses a lightweight, deterministic relational scoring model rather than machine learning. It compares a student's skills, role preferences and goals with the requirements of a role slot, so the same input always gives the same, explainable match percentage. This approach is easy to test, needs no training data and runs directly on the existing database. Overall, the design and architecture focus on speed and practicality.

- **Organisational context:**

SCoop is developed by a team of four students as part of a university course. Some skills and experience are missing in the team, so part of the work involves learning new technologies while building the product. The project is carried out in parallel with other courses and exams, which limits the time each member can spend on it every week. The team has one fixed organisational position, the Project Manager, who coordinates communication, plans and assigns tasks, tracks progress and collects feedback from the course supervisor. The other members have flexible roles that change from task to task, moving between development, design, testing and documentation. The full project, from concept to a working MVP, has to be completed within one semester of 15 weeks. Day to day communication and quick decisions take place in a Telegram group chat. The supervisor reviews progress regularly and can correct the team's direction.

- **Legal context:**

SCoop is developed in Poland, so it must follow both European Union and Polish law, with the General Data Protection Regulation (GDPR, known in Poland as RODO). The platform stores personal data such as names, university email addresses, skills, organisation memberships and peer reviews, so every piece of

data must be kept secure. Users must be able to view, correct, export and delete their data, and when an account is removed, personal information has to be deleted or anonymised. Because some students may be younger than 18, the terms of service will require users to be at least 16, which is the age from which a person in Poland can consent to data processing on their own. Peer reviews directly affect a student's reputation, so the platform needs clear rules for acceptable content and a way to report false or offensive reviews. SCoop only connects people and does not claim any ownership of project ideas or code; intellectual property is agreed within each team, and this will be stated in the terms of service. The platform will also ask for consent before using non-essential cookies and will send only account related emails, such as verification and team invitations. All technologies used are open source with permissive licences and external services that process user data on SCoop's behalf, such as hosting and email providers, will be checked for GDPR compliance, preferably storing data within the EU.

1.7. Technologies used in the project

Justify the choice of technology. Focus on each element and provide the required information

- technology name,
- description of the technology,
- justification,
- key responsibilities in the project, e.g., front-end development,
- link to the technology description.

1. **Django**

- Technology Name: Django (Python Web Framework)
- Description of the Technology: A high-level Python web framework that encourages rapid development and clean, pragmatic design.
- Justification: Django is ideal for an MVP because it comes with built-in user authentication, an Object-Relational Mapper, and a ready-to-use admin panel. Therefore it allows to manage student profiles, monitor role slots, and view database entries without writing all the code from scratch.
- Key Responsibilities in the Project: Handling server-side routing, user authentication, processing form submissions like project creation and team applications, and managing database operations.
- Link to Technology Description: Django Documentation

2. **PostgreSQL**

- Technology Name: PostgreSQL
- Description of the Technology: Relational database management system allowing to handle complex data relationships and strong integrity constraints.
- Justification: Perfectly matches Django's built-in ORM to handle project's relational requirements and store all crucial application data.
- Key Responsibilities in the Project: Secure data persistence, storing student profiles and hackathon data, tracking project applications, and logging teammate ratings.

- Link to Technology Description: PostgreSQL Documentation

3. **Tailwind CSS**

- Technology Name: Tailwind CSS
- Description of the Technology: A utility-first CSS framework that is designed to build modern user interfaces directly inside HTML or templates.
- Justification: Integrates smoothly with Python web templates to help quickly style dashboards, match percentage progress bars , and role-casting cards without building custom CSS files.
- Key Responsibilities in the Project: Page layout design, responsive mobile/desktop styling, and UI component styling.
- Link to Technology Description: Tailwind CSS Documentation

4. **Resend (Python SDK)**

- Technology Name: Resend
- Description of the Technology: An email API service with a straightforward Python library for sending transactional emails.
- Justification: Essential for project's Reputation & Trust feature. It allows Python backend to reliably trigger and send university email verification tokens to .edu addresses.
- Key Responsibilities in the Project: Sending account verification emails and team-invite notifications to registered students.
- Link to Technology Description: Resend Python Documentation

5. **GitHub**

- Technology Name: GitHub
- Description of the Technology: A cloud-based Git version control and collaboration platform for tracking code changes and managing team workflows.
- Justification: Prevents code conflicts, allows team members to work on separate features (e.g., matching algorithm vs. profile UI) in separate branches, and uses Pull Requests to review code before merging. Speeds up collaborative development in general.
- Key Responsibilities in the Project: Version control, collaborative code reviews, issue tracking, and hosting project repositories.
- Link to Technology Description: GitHub Website